

Stateless vs. Stateful Aggregation in Orbital Federated Learning: Design Space and Hybrid Policy

Xiao Luo

Department of Computer Science, Purdue University
CS 59200-EAI research project proposal
Instructor: Prof. Bharat Bhargava

September 15, 2026

This project is Chapter 2 of my preliminary thesis proposal, scoped down to one semester. Section 5.2 maps the chapter's first phase onto the three course deliverables.

1 Topic and problem definition

LEO constellations are becoming computing platforms. A satellite sees a given ground station for 5 to 15 minutes per ~ 90 -minute orbit, and raw-data downlink no longer keeps pace with onboard sensing, so federated learning (FL) is the natural response: train locally, ship compact model updates. But today's satellite-FL frameworks inherit a substrate built for datacenters. They assume an always-on parameter server and a persistent training runtime on every satellite, which between contacts holds memory, drains harvested solar energy, and contends with the primary mission payload. Serverless (FaaS) execution, event-driven and scale-to-zero, fits the orbital duty cycle far better. My thesis argues that serverless is the right systems abstraction for orbital FL provided cold-start scheduling, aggregation-state placement, and state migration are redesigned around orbital determinism. This project takes the second of the three.

FL aggregation is inherently stateful: the global model and round metadata must survive across rounds. FaaS functions are canonically stateless and externalize their state. In orbit there is nowhere stable to externalize it to. ISL bandwidth is scarce, nodes churn as visibility changes, and the energy budget follows the eclipse cycle. Li et al. measured that budget on two orbiting SmallSats over 95 and 123 days [17]. Each orbit yields roughly 60 minutes of harvesting followed by 30 minutes on battery; harvested power sits in the tens of watts; a protective 50% depth-of-discharge (DoD) cap shuts down every power-consuming payload once crossed; and a compute payload left running continuously drains the battery within tens of hours. Those numbers are the envelope this project has to respect.

Four placements are on the table: stateless, where each aggregator invocation re-fetches the global model from a replicated store; stateful, with state pinned on sink satellites; hierarchical, aggregating intra-plane then inter-plane; and gossip, a stateless exchange between ISL neighbors. Nobody has quantified this design space for FL under orbital churn. Problem definition: given a constellation topology, a stream of client updates, and per-node energy budgets, choose the placement pattern for each aggregation point in each round so that training reaches target accuracy inside the round deadline at minimum cost in energy, bytes transferred, staleness, and recovery time. Hypothesis: a measurable regime boundary separates the four, and a hybrid policy that switches patterns per round beats every fixed one.

2 State of the art: what has been done and what remains

Three literatures bracket this question. None of them answers it.

Serverless FL on terrestrial clouds. FedLess [8] established FaaS-based FL across heterogeneous providers; FedLesScan [9] added clustering-based semi-asynchronous straggler mitigation; Apodotiko [10] contributed scoring-based client selection under hardware heterogeneity; just-in-time aggregation [12] made the server itself serverless at over 90% resource reduction; and LIFL [11], the current state of the art, uses eBPF and shared-memory hierarchical aggregation to hit 70% accuracy $1.68\times$ faster with $4.23\times$ fewer CPU cycles. Every one of them assumes a reachable cloud control plane, stable bandwidth, and a state store that is always one RPC away.

Satellite FL without serverless. FedSpace [1] schedules aggregation around deterministic orbital connectivity and formalizes the idleness-versus-staleness trade-off, cutting training time 38.6%. Razmi et al. add ground-assisted asynchronous FL [2] and energy-aware scheduling that triples battery lifetime without convergence loss [3]. FedLEO/AsyncFLEO [4] exploit intra-orbit propagation and sink satellites for up to $29\times$ speed-up; FedSN [5] and SatFed [6] attack heterogeneity and constrained links; Kim et al. [7] swept 768 configurations of off-the-shelf FL in orbit. All of them fix where the parameter server lives before the experiment starts. Placement is an input, never a variable.

Serverless on the LEO edge, without FL. Komet [13] pairs FaaS with a replicated state store and argues for compute/storage decoupling; Krios [14] adds proactive orchestration abstractions; Trabant [19] adds multi-tenant FaaS under energy and thermal envelopes; Databelt [15] propagates and fuses generic workflow state across the 3D continuum; Celestial [16] supplies the microVM testbed I build on. None of them runs FL: no aggregation, no rounds, no convergence concern.

So the ingredients exist and the measurement does not. What is missing is a head-to-head comparison of the four patterns for FL specifically, under Shell-1-scale topologies, eclipse-driven energy budgets, and node churn, scored on energy, bandwidth, staleness, and fault tolerance, and ending in a placement policy rather than a catalogue of options.

3 Proposed technique

I will build OrbitState, a serverless aggregation runtime that treats aggregation state as a first-class object with an explicit placement. Its central abstraction, AggState, carries the global model, the round’s partial aggregates, and round metadata, and exposes four operations: fetch, merge, checkpoint, hand-over. The four placements then become four policies behind one interface, and client and aggregator functions run unmodified under any of them.

Two design decisions make switching safe rather than merely convenient. First, I structure the aggregation step as a commutative monoid over (weighted partial sum, sample count) pairs, so partial aggregates merge in any order at any node and the round result does not depend on which pattern produced it. Second, an energy ledger tracks each node’s harvested and consumed energy and its running DoD, calibrated against the telemetry traces released with [17] and the energy-aware FL results in [3]. Pattern selection lives in the control plane, driven by SGP4-propagated visibility and ISL schedules computed from public TLEs. Update transfer and merge live in the data plane.

The hybrid selector is the algorithmic contribution. Once per aggregation point per round, it reads a feature vector describing the window ahead: contact length, ISL chain persistence, eclipse phase and battery state, churn rate, pending update volume. I formalize the choice as a constrained

optimization, minimizing a weighted cost over energy, bytes, and staleness subject to the round deadline and a long-run DoD constraint, with hysteresis so the policy does not flap between patterns. Section 5.1 gives the rule.

4 How this work advances the solution

Every satellite-FL paper pins the aggregator somewhere; every serverless-FL paper assumes a cloud store. I turn that fixed assumption into a per-round decision and measure what each choice costs. Two things come out. The first is a regime map of aggregation placement under orbital churn and eclipse energy, which does not currently exist for FL. The second is a policy with a correctness argument behind it: because merges commute, switching patterns changes what a round costs, never what it computes. The selector is also the piece my other two chapters need, since a cold-start scheduler has to know where state will be fetched from and a migration policy has to know which copy is authoritative.

I expect three objections and have designed against each. “Measurement only” is answered by the formalized selector and the fault-injection experiments, not by more plots. “The conclusion is obvious” is answered by the matrix itself, which is built to test whether the ranking flips once staleness and eclipse energy enter the cost function together; no prior result covers that corner. “This is generic state management” is answered by the cost function, which carries round deadlines and the coupling between staleness and accuracy that a generic service simply does not have.

5 Algorithm, experiments and data, and expected conclusions

5.1 Algorithm: the hybrid selector

Inputs: the SGP4-derived contact and ISL schedule, the energy ledger, a churn estimate, and a per-pattern cost table built offline from the trade-off matrix and tuned online. At each decision epoch t (a round, or a contact window), for each aggregation point:

1. Build the feature vector x_t : remaining window length, ISL chain persistence, eclipse flag, DoD D_t , churn rate, pending update bytes.
2. For each pattern $p \in \{\text{stateless}, \text{stateful}, \text{hierarchical}, \text{gossip}\}$, look up expected energy $E_p(x_t)$, bytes $B_p(x_t)$, staleness $S_p(x_t)$, and recovery cost $R_p(x_t)$.
3. Among the patterns that meet the round deadline, choose

$$p^* = \arg \min_p \left[Q_t E_p + V(w_B B_p + w_S S_p + w_R R_p) \right],$$

where Q_t is a virtual queue that grows with DoD overshoot above the cap (the drift-plus-penalty treatment [20] that [17] applies to the same battery constraint) and V sets the trade between cost and energy safety. Switch away from the current pattern only when the gain clears a hysteresis margin.

4. Run the round under p^* : clients push updates, partial aggregates merge through the monoid merge, and AggState is checkpointed with application-level persistence under that pattern’s rules.
5. Update Q_t from the ledger, and update the cost table from observed costs by exponential moving average.

The bar is not just the four pure patterns. The selector also has to beat the fixed hybrid “stateless on visibility passes, stateful on persistent ISL chains”.

5.2 Experiments and data

Testbed. Celestial [16] emulates satellites as microVMs on one workstation with one GPU; Knative runs the client and aggregator functions; Hypatia/StarryNet supplies network traces; Starlink Shell-1 TLEs drive the scenarios. I evaluate at three fidelity tiers: L1 full-stack emulation (10 to 40 nodes), L2 lightweight-runtime emulation (100 to 300 nodes), and L3 trace-driven simulation at Shell-1 scale (about 1,584 satellites). L1 is cross-validated against L2 and L3 on shared scenarios, with the agreement criterion fixed before those runs. I calibrate the energy model on the traces from [17]; if the schedule allows, one hardware-in-the-loop measurement on a Jetson or Raspberry Pi anchors it to real hardware.

Workloads. MNIST with a small CNN for cheap sweeps, CIFAR-10 with ResNet-18 for update-size pressure, and EuroSAT [18] with a light CNN as the Earth-observation workload. All three sit inside the single-workstation budget our group used for STARFed [21].

Matrix and baselines. Pattern $\times$ constellation scale $\times$ churn level $\times$ non-IID level, with fault injection (satellite loss, link drops). Baselines: each pure pattern; a pinned-server configuration in the style of FedSpace and FedLEO; LIFL-style hierarchical aggregation as the serverless efficiency reference; a Komet-style replicated store.

Data collected. Time to target accuracy, wall-clock per round, aggregation p50/p95 latency, staleness distribution, bytes per useful update, recovery time after link loss or node failure, state read/write amplification, and energy-normalized accuracy gain. Every run emits NDJSON telemetry under a schema that maps each symbol in the cost function to a measurement point, so the offline cost table and the reported results come from the same source.

Semester plan. Intermediate report (Due on Oct 7): the L1 pipeline running all four patterns on 10 to 40 emulated nodes, the energy ledger validated against [17], the fault-injection harness, and a first slice of the matrix (MNIST and CIFAR-10, two churn levels). Final report and presentation: EuroSAT, the full matrix at L2 plus a Shell-1-scale L3 run, the selector derived from the matrix and validated on held-out scenarios against every fixed policy, and the regime map with guidelines.

5.3 Expected conclusions and guidelines

The deliverable is a regime map: which pattern wins under which conditions of contact length, ISL persistence, eclipse phase, churn, and non-IID level, with each pattern’s cost per useful update in energy, bytes, and staleness. From that map come placement guidelines for anyone building FL on a constellation, plus the selector itself as a component the rest of my thesis reuses. The claim is falsifiable, deliberately. If the selector fails to beat the best fixed policy by more than seed variance without losing final accuracy, I report the design-space characterization and the regime map instead, which is still the chapter’s core result.

References

- [1] J. So, K. Hsieh, B. Arzani, S. Noghabi, S. Avestimehr, R. Chandra. FedSpace: An efficient federated learning framework at satellites and ground stations. arXiv:2202.01267, 2022.

- [2] N. Razmi, B. Matthiesen, A. Dekorsy, P. Popovski. On-board federated learning for dense LEO constellations. *IEEE ICC*, 2022.
- [3] N. Razmi, B. Matthiesen, A. Dekorsy, P. Popovski. Energy-aware federated learning in satellite constellations. *IEEE GLOBECOM Workshops*, 2024.
- [4] M. Elmahallawy, T. Luo. FedLEO / AsyncFLEO: Communication-efficient federated learning for LEO satellite constellations. *IEEE ICC*, 2023.
- [5] Z. Lin et al. FedSN: A federated learning framework over heterogeneous LEO satellite networks. *arXiv:2311.01483*, 2024.
- [6] Y. Zhang et al. SatFed: A resource-efficient LEO satellite-assisted heterogeneous federated learning framework. *arXiv:2409.13503*, 2024.
- [7] G. Kim, F. Svoboda, N. Lane. Bringing federated learning to space. *IEEE Aerospace Conference*, 2026.
- [8] A. Grafberger, M. Chadha, A. Jindal, J. Gu, M. Gerndt. FedLess: Secure and scalable federated learning using serverless computing. *IEEE BigData*, 2021.
- [9] M. Elzohairy et al. FedLesScan: Mitigating stragglers in serverless federated learning. *IEEE BigData*, 2022.
- [10] M. Chadha, A. Jensen, J. Gu, O. Abboud, M. Gerndt. Apodotiko: Enabling efficient serverless federated learning in heterogeneous environments. *IEEE/ACM CCGrid*, 2024.
- [11] S. Qi, K. K. Ramakrishnan, M. Lee. LIFL: A lightweight, event-driven serverless platform for federated learning. *arXiv:2405.10968*, 2024.
- [12] K. R. Jayaram, A. Verma, G. Thomas, V. Muthusamy. Just-in-time aggregation for federated learning. *arXiv:2208.09740*, 2022.
- [13] T. Pfandzelter, D. Bermbach. Komet: A serverless platform for low-earth orbit edge services. *ACM SoCC*, 2024.
- [14] V. Bhosale, A. Gavrilovska, K. Bhardwaj. Krios: Scheduling abstractions and mechanisms for enabling a LEO compute cloud. *ACM SoCC*, 2024.
- [15] C. Marcelino, L. Guelmino, T. Pusztai, S. Nastic. Databelt: A continuous data path for serverless workflows in the 3D compute continuum. *arXiv:2508.15351*, 2025.
- [16] T. Pfandzelter, D. Bermbach. Celestial: Virtual software system testbeds for the LEO edge. *ACM/IFIP Middleware*, 2022.
- [17] Q. Li et al. Exploring real-time satellite computing: From energy and thermal perspectives. *IEEE RTSS*, 2024.
- [18] P. Helber, B. Bischke, A. Dengel, D. Borth. EuroSAT: A novel dataset and deep learning benchmark for land use and land cover classification. *IEEE JSTARS*, 2019.
- [19] T. Pfandzelter et al. Trabant: A serverless architecture for multi-tenant orbital edge computing. *arXiv:2504.08337*, 2025.
- [20] M. J. Neely. Stochastic network optimization with application to communication and queueing systems. *Synthesis Lectures on Communications*, 2010.
- [21] Z. Liu, B. K. Bhargava, N. Aneja. STARFed: Link-aware defense against poisoning attacks in satellite-terrestrial federated learning. *IEEE TNSE*, in submission.